

Assignment 1

Part A

```
Q.1  BEGIN
      DECLARE N : INTEGER
      INPUT N
      FOR i = N STEP - 1
        PRINT i
      END FOR
    END
```

```
Q.2  BEGIN
      DECLARE Year : INTEGER
      INPUT Year
      IF (year MOD 4 = 0 AND year MOD 100 != 0) OR (year MOD 400 = 0)
        PRINT "Leap Year"
      ELSE
        PRINT "Not a Leap year"
      ENDIF
    END
```

Q.3 BEGIN

DECLARE num, temp, count : INTEGER

INPUT num

temp = ABS(num)

ABS = removes sign

count = 0

IF temp == 0 THEN

count = 1

ELSE

WHILE temp > 0 DO

temp = temp / 10

count = count + 1

ENDWHILE

PRINT "Total digits:", count

IF count == 1 THEN

PRINT "Single digit number"

ELSE IF count == 2 THEN

PRINT "Double digit number"

ELSE

PRINT "Triple digit number or more"

ENDIF

ENDIF

END

Q.4 BEGIN

```
DECLARE num, multiple : INTEGER
FOR num = 1 TO 10 DO
    IF num MOD 3 == 0 THEN
        Continue
    ENDIF
    PRINT " Table of " + num
    FOR multiplier = 1 TO 10 DO
        PRINT num + "x" + multiplier + "=" + (num * multiplier)
    ENDFOR
ENDFOR
END
```

Q5

BEGIN

```
DECLARE num, current, temp, sum : INTEGER
INPUT num
current = ABS(num)
WHILE current > 0 DO
    sum = sum + (temp MOD 10)
    temp = temp / 10
ENDWHILE
PRINT "Intermediate Sum", sum
current = sum
PRINT "Final Single Digit", current
```

END

Part B

Q.1 Algo

1. Start

2. Initialize total revenue = 0

3. Input integer N (number of guests).

Input

- N
- season
- room type
- nights

Process

1. Loop N times
2. Select rates based on season & room type
3. Compute subtotal discount, and net total
4. Accumulate revenue

Output

- Guest Final Price
- Total hotel revenue

4. For $i = 1$ to N :

- Input season, timings (peak season / off peak), room type and stay nights
- If season == peak then charge higher rates
- $\text{subtotal} = \text{rate} * \text{nights}$
- If $\text{nights} > 7$: $\text{discount} = \text{subtotal} * 0.15$, else $\text{discount} = 0$
- $\text{final price} = \text{subtotal} - \text{discount}$

PAC

Data Input	Formula	output
N , season, room-type, nights	$\text{subtotal} = \text{rate} * \text{nights}$	final price per guest
	$\text{discount} = \text{subtotal} * 0.15$	total revenue
	$\text{final price} = \text{subtotal} - \text{discount}$	
	$\text{total revenue} += \text{final price}$	

Pseudocode

totalrevenue = 0

INPUT N

FOR $i = 1$ TO N DO

INPUT season, roomtype, nights

IF season "Peak" THEN

IF roomtype == "Standard" THEN rate = 5000

ELSEIF roomtype == "Deluxe" THEN rate = 8000

ELSEIF roomtype == "Suite" THEN rate = 12000

ENDIF

ELSE

IF roomtype == "Standard" THEN rate = 3000

ELSEIF roomtype == "Deluxe" THEN rate = 5000

ELSEIF roomtype == "Suite" THEN rate = 8000

ENDIF

subtotal = rate * nights

IF nights > 7 THEN

discount = subtotal * 0.15


```
ELSE  
    discount = 0
```

```
ENDIF
```

```
final price = subtotal - discount +
```

```
total revenue = total revenue + final price
```

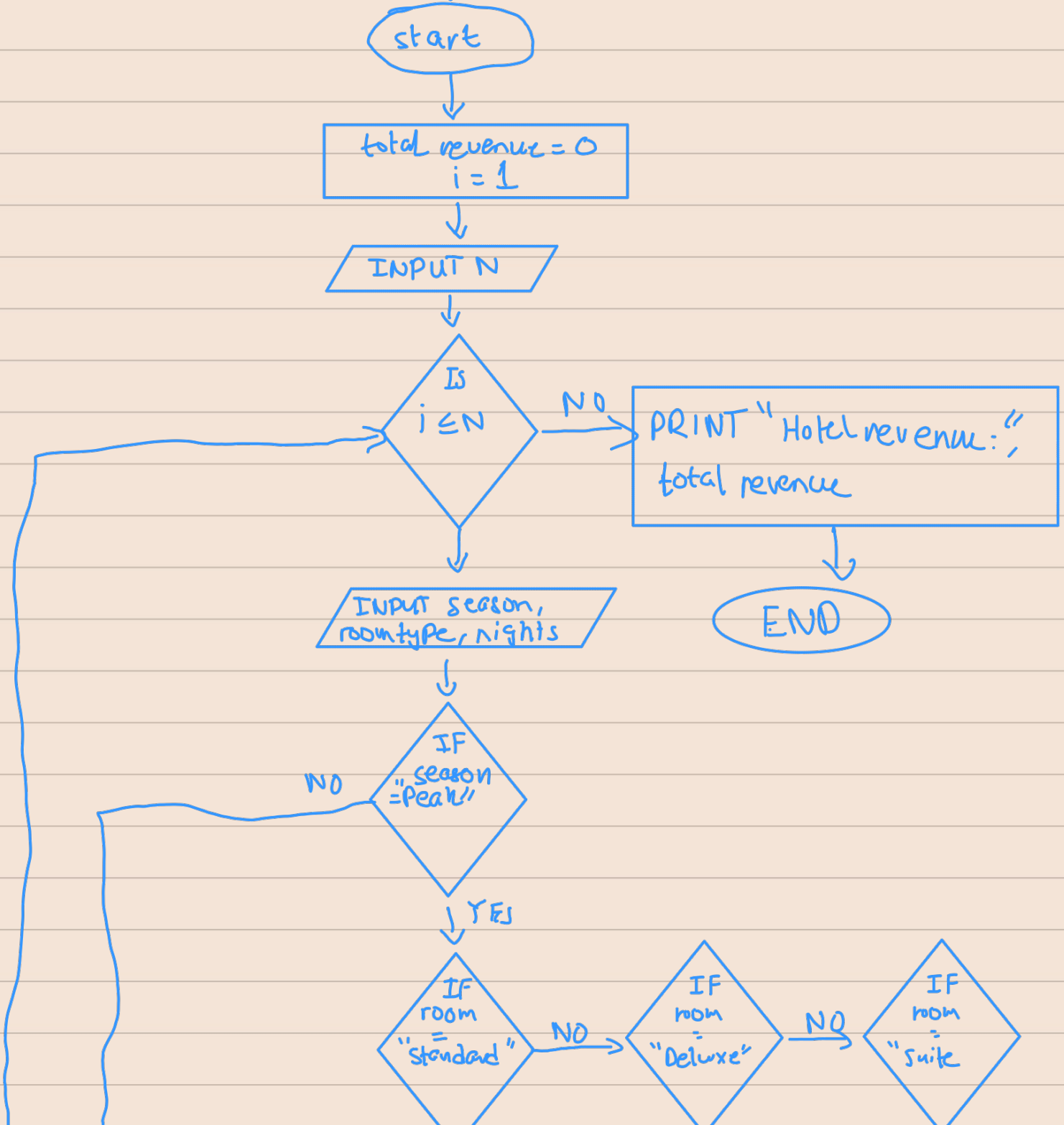
```
PRINT "Guest", i " Final price", final price
```

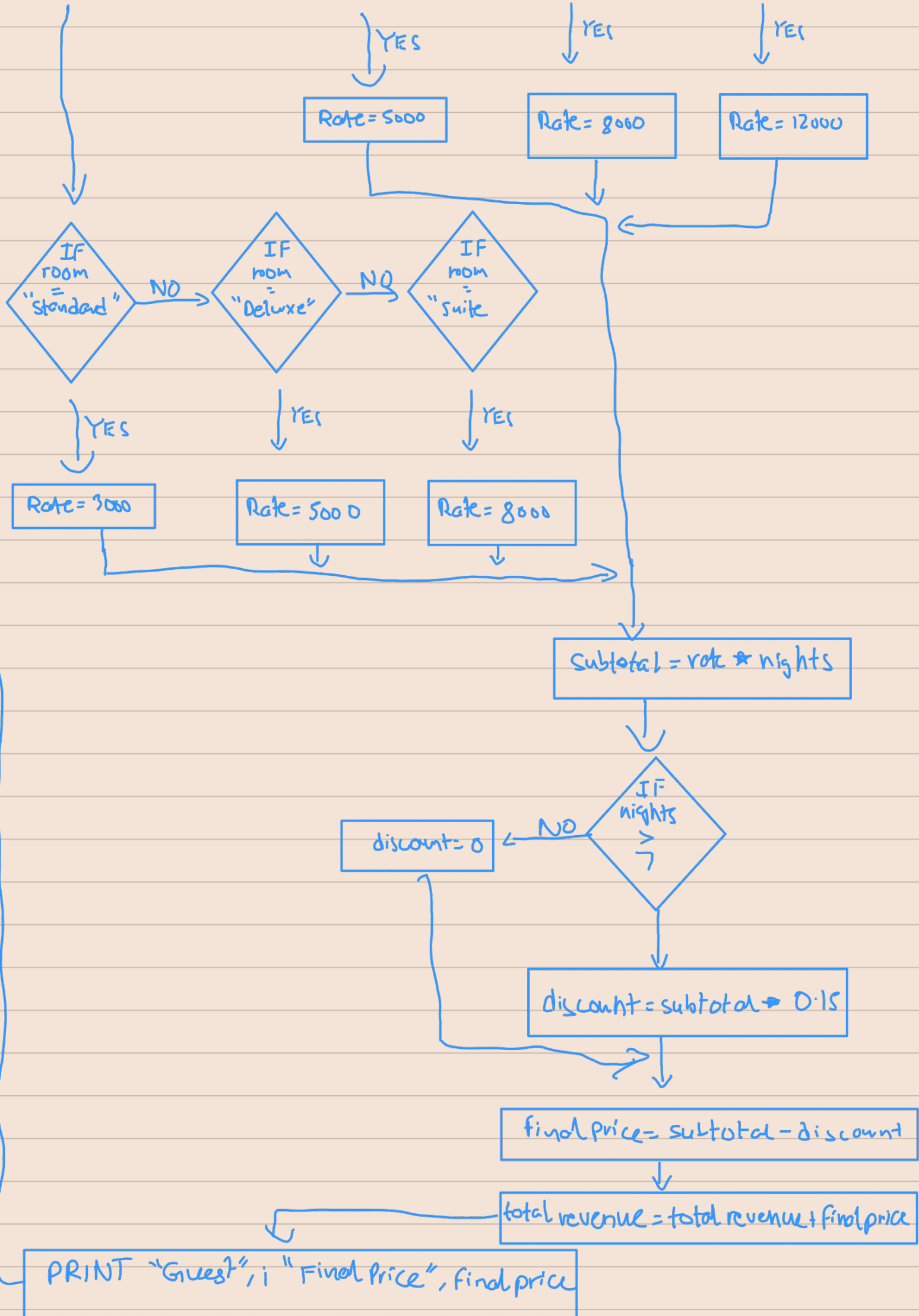
```
ENDFOR
```

```
PRINT "Hotel revenue", total revenue
```

```
END
```

Flow chart





Q.2 Algo

1. Start

2. $\text{currentFloor} = 0$

3. Total requests (N)

4. Loop $i = 1$ to N

- Read requested floor

- If requested floor $>$ current floor: Print "moving up"

- Else If requested floor $<$ current floor: Print "moving down"

- Else Print "Doors Opening"

- $\text{currentFloor} = \text{requested floor}$

5. End

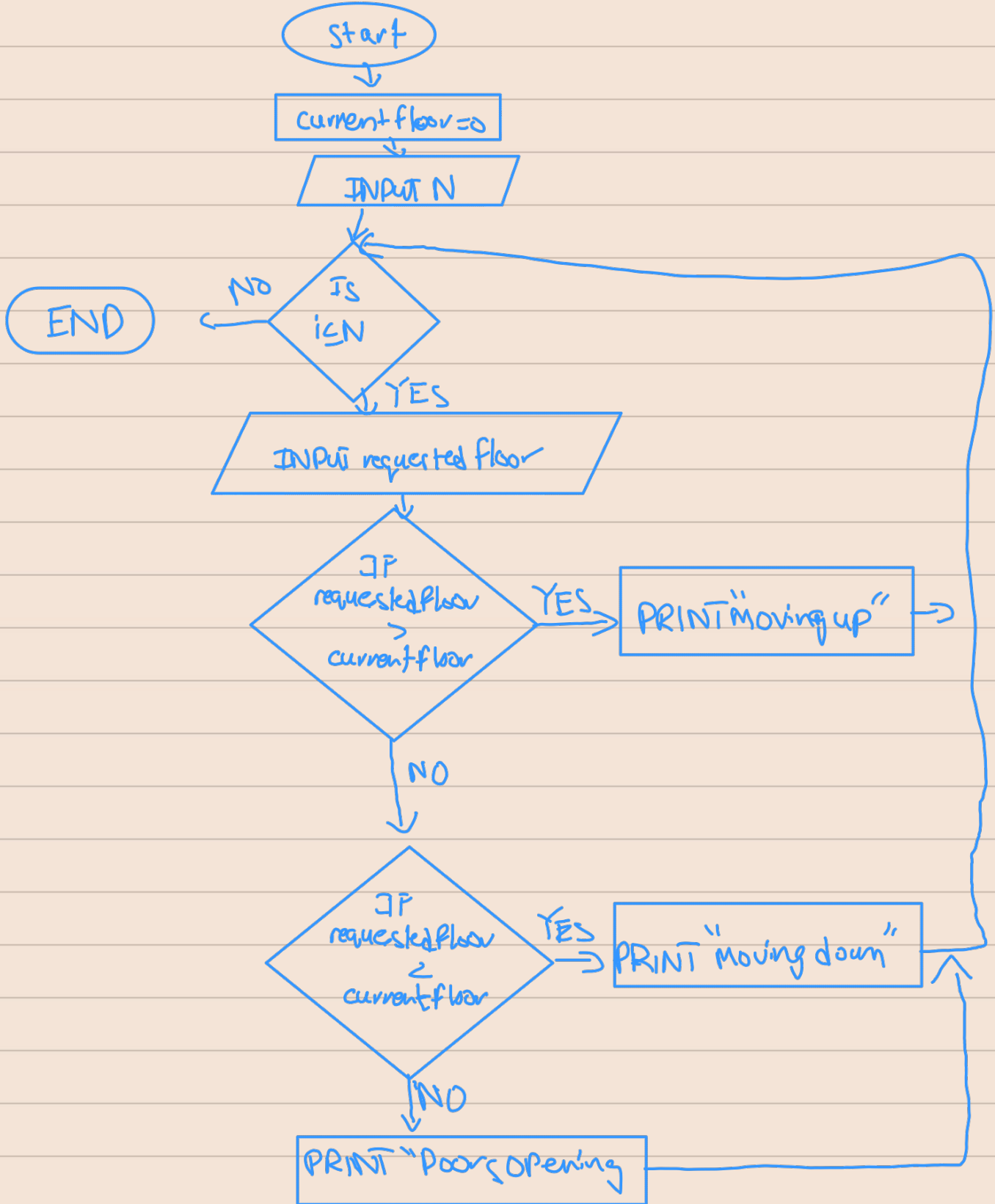
DAC

Input	Formula	Output
N, requested floor	• $\text{requested floor} = \text{current floor}$ • compare requested floor with current floor	"Moving Up" "Moving down" "Doors opening"

IDO

Input	Process	Output
• Total request N • Floor requests	compare target floor vs current position and update status	Elevator movement message

Flow chart



Pseudocode

BEGIN

DECLARE current floor, requested floor, N, i : INTEGER

current floor = 0

INPUT N

FOR i = 1 TO N DO

INPUT requested floor

IF requested floor > current floor THEN

PRINT "Moving up"

ELSE IF requested floor < current floor THEN

PRINT "Moving down"

ELSE

PRINT "Doors Opening"

ENDIF

current floor = requested floor

ENDFOR

END

Q.3 Algo

1. INPUT total students N

2. Loop $i = 1$ to N

- Set $sum = 0$, $deficiency = FALSE$

- Loop $j = 1$ to S :

- INPUT mark

- $sum = sum + mark$

- IF $mark < 33$: $deficiency = TRUE$

- $average = sum / S$

- ELSE IF $average \geq 80$: Print "distinction"

- ELSE IF $average \geq 60$: Print "Pass"

- ELSE: Print Fail

PAC

Input

N , S marks per student

Formula

- $sum = \text{sum of marks}$

- $average = sum / S$

- Check individual mark

< 33

IPD

Input

- N

- S marks

Process

compute average, evaluate

subject mark floor constraint,
evaluate result

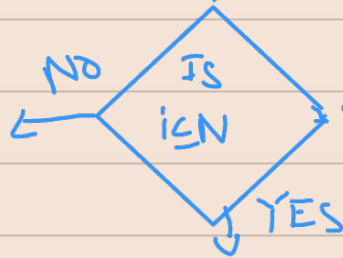
Output

student Average
and performance
status.

Flowchart

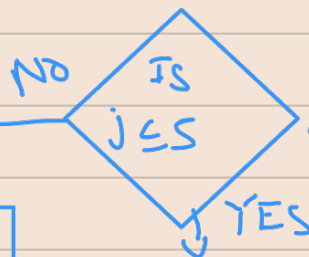
Start

INPUT N



END

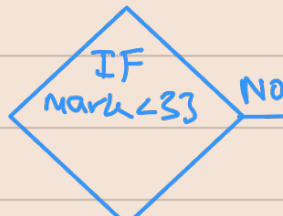
sum = 0, deficiency = FALSE



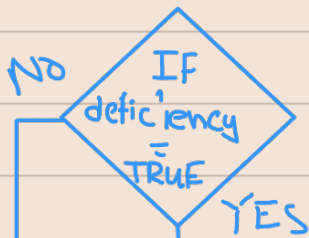
average = sum / 5

INPUT mark

sum += mark
 $j++$

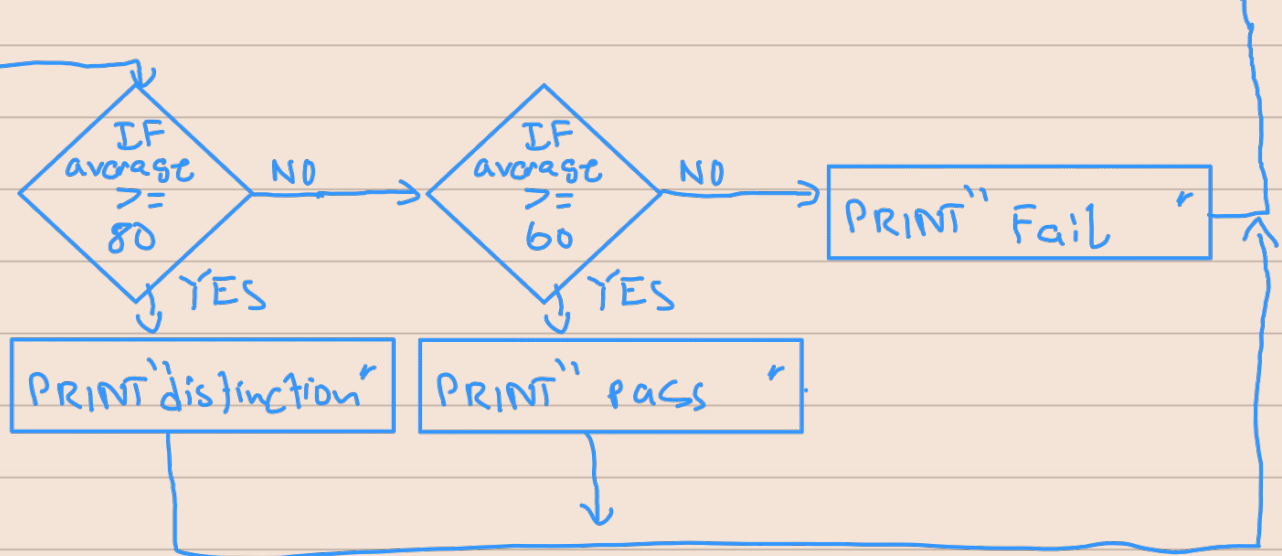


deficiency = TRUE



PRINT "Fail subject deficiency"

$i++$



Pseudo Code

Start

```

DECLARE N,i,j: INTEGER
DECLARE average: REAL
DECLARE deficiency: BOOLEAN
INPUT N
FOR i = 1 TO N DO
    sum = 0
    deficiency = FALSE
    FOR j = 1 TO 5 DO
        INPUT mark
        sum = sum + mark
        IF mark < 33 THEN
            deficiency = TRUE
        ENDIF
    ENDFOR
    average = sum / 5
    Print "student has an average of", average
    IF deficiency THEN
        PRINT "Fail Subject deficiency"
    ELSEIF average >= 80 THEN
        PRINT "Distinction"
    
```



```
ELSEIF average >= 60 THEN  
    PRINT " Pass      "
```

```
ELSE  
    PRINT " Fail "
```

```
ENDIF
```

```
ENDFOR
```

```
END
```

Q.4 Algo

1. INPUT quantity, price, discount, tax
2. IF quantity ≤ 0 OR price ≤ 0 OR discount ≤ 0
 - Print "Invalid Input"
 - Stop program
4. Calculus subtotal $s = q \times p$
5. calculate discounted amount $a = s - \frac{s \times d}{100}$
6. calculate final bill $f = a + \frac{a \times t}{100}$
7. Display items

PAC

Input	Formula	Output
quantity, price, discount, tax	• data validation • $s = q \cdot p$ $a = s - \left(\frac{s \cdot d}{100} \right)$ $f = a + \left(\frac{a \cdot t}{100} \right)$	Error message OR Program terminated

IPO

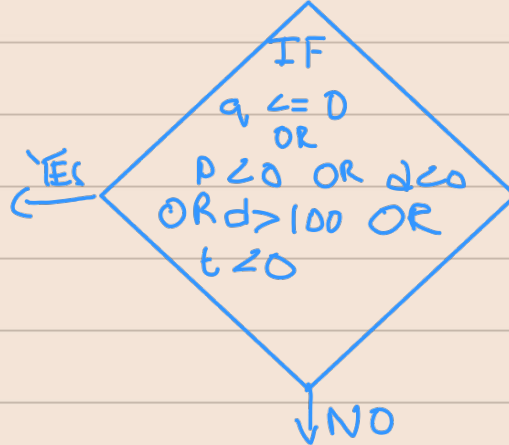
Input	Process	Output
quantity, price, discount %, tax %	validate inputs, calculate mathematically values	Item wise bill and Final amount

Flow chart

start



Input q, p, d, t



$S = q \times p,$



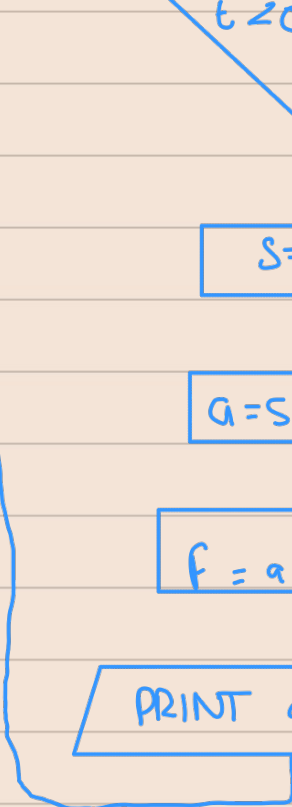
$a = S - (S \times d) / 100$



$f = a + (a \times t) / 100$



PRINT q, p, S, d, f



END

Pseudocode

Start

DECLARE q : INTEGER

DECLARE p, d, t, s, da, F : REAL

INPUT q, p, d, t

IF $q \leq 0$ OR $p < 0$ OR $d < 0$ OR $d > 100$ OR $t < 0$ THEN

PRINT "Enter correct Values"

ENDIF

$S = q * p$

$da = S - (S * d / 100)$

$F = da + (da * tax / 100)$

PRINT "Shopping list"

PRINT "Quantity", q

PRINT "price of item", price

PRINT "subtotal", s

PRINT "discount", da

PRINT "Final amount", F

END

Q5 Algo

1. Set capacities $capA=20$, $capB=40$, $capC=15$

2. Set initial used space = $ocA=0$, $ocB=0$, $ocC=0$, $cars=0$, $bikes=0$, $vans=0$, $accepted=0$, $rejected=0$

3. Input total-expected-vehicle

4. $i=1$ to total-expected-vehicle:

- Input and validate loop for $v_type('C','B','V')$, category ('F','S','G'), permit ('Y','N'). Re prompt on invalid input.

- IF $permit == 'N'$

- Input emergency flag $is_emergency$

- IF $is_emergency == 'N'$: Reject vehicle, increment rejected, Next vehicle

- Evaluate Parking Assignment

- IF $v_type == 'V'$: IF $ocA+1 \leq capA$, assign zone A. ELSE reject

- ELSE IF $ocA+1 \leq capA$, assign zone A. ELSE reject

- Student ('S'):

- IF $v_type == 'V'$: IF $ocC+1 \leq capC$, assign zone C. ELSE reject

- ELSE IF $ocB+1 \leq capB$, assign zone B. ELSE reject

- Visitor ('G'):

- IF $v_type == 'V'$: IF $ocC+1 \leq capC$, assign zone C. ELSE reject

- ELSE IF $ocC+1 \leq capC$, assign zone C. ELSE reject

- IF Accepted:

- Increment accepted. update appropriate zone occupancy and vehicle counter.

- Print assigned zone & remaining space.

5. Display Parking Summary

6. End

PAC

Input	processing/Formula	Output
v_type, category, permit, is_emergency.	• Validation using loops • Capacity availability checks • Occupancy tracking and allocation logic	Zone allocation, Rejection Cause, Campus Summary Report.
IPO		

Input	Process	Output
Vehicle parameters & emergency state	Validate inputs - checks authorization $\rightarrow$ Verify zone space $\rightarrow$ update stats.	Allocation result, occupancy logs, campus-wide capacity summary.

PSEUDOCODE

START

DECLARE capA, capB, capC, occA, occB, occC AS INTEGER

DECLARE cars, bikes, vans, accepted, rejected, total AS INTEGER

DECLARE v_type, category, permit, emergency AS CHAR

capA = 20, capB = 40, capC = 15

occA = 0, occB = 0, occC = 0

cars = 0, bikes = 0, vans = 0, accepted = 0, rejected = 0

INPUT total

FOR i = 1 TO total DO

REPEAT

INPUT v_type, category, permit

UNTIL (v_type IN ['C', 'B', 'V']) AND (category IN ['F', 'S', 'G']) AND (permit IN ['Y', 'N'])

emergency = 'N'

IF permit = 'N' THEN

REPEAT

INPUT emergency

UNTIL (emergency IN ['Y', 'N'])

```
IF emergency == 'N' THEN
```

```
    PRINT "Rejected no permit"
```

```
    rejected = rejected + 1
```

```
ENDIF
```

```
ENDIF
```

```
IF category == 'F' THEN
```

```
    IF occA + 1 <= capA THEN
```

```
        occA += 1
```

```
        accepted += 1
```

```
        IF v_type == 'C' THEN
```

```
            cars += 1
```

```
        ELSEIF v_type == 'B' THEN
```

```
            bikes += 1
```

```
        ELSEIF v_type == 'V' THEN
```

```
            vans += 1
```

```
        ENDIF
```

```
        PRINT "Assigned to Zone A. Remaining Space: ", (capA - occA)
```

```
    ELSE
```

```
        PRINT "Rejected zone A Full
```

```
        rejected += 1
```

```
    ENDIF
```

```
ELSEIF category == 'S' THEN
```

```
    IF v_type == 'V' THEN
```

```
        IF occC + 1 <= capC THEN
```

```
            occC += 1
```

```
            accepted += 1
```

```
            vans += 1
```

```
            PRINT "Assigned to zone C. Remaining Space: ", (capC - occC)
```

```
        ELSE
```

```
            PRINT "Rejected: zone C Full for Van "
```

```
            rejected += 1
```

```
        ENDIF
```

ELSE

IF $occB + 1 \leq capB$ THEN

$occB += 1$

$accepted += 1$

IF $v_type == 'C'$ THEN

$cars += 1$

ELSEIF $v_type == 'B'$ THEN

$bikes += 1$

PRINT "Assigned to zone B. Remaining space: ", $(capB - occB)$

ELSE

PRINT "Rejected: zone B Full for Van "

$rejected += 1$

ENDIF

ELSEIF $category == 'G'$ THEN

IF $v_type == 'V'$ THEN

IF $occC + 2 \leq capC$ THEN

$occC += 2$

$accepted += 1$

$vans += 1$

PRINT "Assigned to zone C. Remaining space: ", $(capC - occC)$

ELSE

PRINT "Rejected, Insufficient space in zone C for Van

$rejected += 1$

ENDIF

ELSE

IF $occC + 1 \leq capC$ THEN

$occC += 1$

$accepted += 1$

IF $v_type == 'C'$ THEN

$cars += 1$

ELSEIF

$bikes += 1$

PRINT "Assigned to zone C. Remaining space:", (cap - occ)

ELSE

PRINT "Rejected: Zone C is full"

rejected = rejected + 1

ENDIF

ENDIF

ENDIF

ENDFOR

PRINT "Total Vehicle processed:", total

PRINT "Total accepted:", accepted, "Total Rejected:", rejected

PRINT "Cars:", cars, "Bikes:", bikes, "Vans:", vans

END